

Architecture Constitution

سند قوانین بنیادین معماری ERP SaaS

Version ۱,۱

۱. هدف سند

این سند به عنوان مرجع اصلی تصمیم‌گیری معماری پروژه ERP SaaS تهیه شده است.

تمامی تصمیمات مربوط به:

- طراحی دیتابیس
- ایجاد جداول
- توسعه ماژول‌ها
- طراحی API
- پیاده‌سازی سرویس‌ها
- توسعه امکانات جدید

باید با اصول این سند منطبق باشند.

هیچ توسعه‌ای نباید صرفاً بر اساس نیاز لحظه‌ای انجام شود و هر تصمیم باید جایگاه خود را در معماری کلان سیستم داشته باشد.

۲. اصل Platform First

این محصول یک Platform است، نه مجموعه‌ای از نرم‌افزارهای مستقل.

معماری کل سیستم بر پایه:

Platform



Core



Modules



Extensions



Integrations

طراحی می‌شود.

تمام قابلیت‌های جدید باید روی این ساختار قرار بگیرند.

۳. اصل Core Only

تمام مفاهیم پایه سیستم فقط یک بار و در Core تعریف می‌شوند.

نمونه:

- Tenant
- User
- Role
- Permission
- Scope
- Organization
- Company
- Branch
- Department
- Audit
- Notification
- Configuration

هیچ ماژولی اجازه ایجاد نسخه موازی از این موجودیت‌ها را ندارد.

۴. اصل Shared Core

تمام ماژول‌ها باید از Core استفاده کنند.

منطق‌های زیر نباید در ماژول‌ها تکرار شوند:

- مدیریت کاربر
- احراز هویت
- کنترل دسترسی
- مدیریت سازمان
- Tenant Management
- Audit

۵. اصل Tenant Isolation By Design

امنیت اطلاعات مشتریان یکی از اصول غیرقابل مذاکره سیستم است.

در معماری: Shared Database Multi-Tenant

- تمام جداول عملیاتی ماژول‌ها باید دارای `tenant_id` باشند.
 - هیچ Query ، API یا Service اجازه دسترسی بدون محدود کردن Tenant را ندارد.
 - هیچ داده‌ای نباید بین Tenant ها قابل مشاهده باشد.
- نشت اطلاعات بین مشتریان (Data Bleeding) یک خطای بحرانی معماری محسوب می‌شود.

۶. اصل Independent Modules

هر ماژول یک Bounded Context مستقل است.

نمونه:

- Accounting
- Inventory
- CRM
- HR
- Payroll
- Manufacturing
- Sales
- Purchase

- Asset Management

هر ماژول:

- مالک منطق کسب و کار خود است.
- ساختار داخلی خود را مدیریت می‌کند.
- مسئول داده‌های دامنه خود است.

۷. اصل No Direct Coupling

وابستگی مستقیم بین Bounded Context ها ممنوع است.

اما این قانون به معنی حذف کامل Foreign Key نیست.

قانون صحیح:

داخل یک: Bounded Context

استفاده از Physical Foreign Key مجاز است.

مثال:

Invoice



Invoice Items



Invoice Payments

بین Bounded Context های مستقل:

Physical Foreign Key ممنوع است.

ارتباط باید از طریق:

- Logical Reference (UUID)

- API

- Event

- Service Contract

باشد.

هدف:

حفظ امکان جداسازی سرویس‌ها و مهاجرت آینده به معماری Microservice.

۸. اصل استاندارد ساختار هر Module

هر ماژول باید حداقل دارای بخش‌های زیر باشد:

۱. Master Data

اطلاعات پایه ماژول.

۲. Transactions

عملیات روزمره.

۳. Documents

اسناد و گردش اطلاعات.

۴. Workflow

فرآیندهای تایید و گردش کار.

۵. Reports

گزارش‌ها و تحلیل‌ها.

۶. Settings

تنظیمات اختصاصی ماژول.

۷. Permission Integration

اتصال به سیستم دسترسی Core.

۸. API / Gateway Layer

تمام ارتباطات بیرونی ماژول باید از یک لایه مشخص API انجام شود.

۹. License & Feature Flag Check

هر ماژول قبل از اجرا باید بررسی کند:

- آیا Tenant این ماژول را خریداری کرده؟
- آیا Feature مربوط فعال است؟
- آیا محدودیت Plan اجازه استفاده می‌دهد؟

۹. اصل Central Authorization

هیچ ماژولی اجازه ایجاد سیستم Authorization مستقل ندارد.

تمام کنترل دسترسی باید از Core انجام شود.

ماژول‌ها فقط مصرف‌کننده خروجی Core هستند.

مبنای کنترل:

User



Role



Permission



Scope



Resource

برای مثال:

کاربر فقط در شعبه مشخص یا انبار مشخص اجازه فعالیت دارد.

۱۰. اصل Scope Driven Access

سیستم Scope باید تمام محدودیت‌های سطح دسترسی را مدیریت کند.

Scope باید قابلیت توسعه برای آینده داشته باشد.

نمونه:

• Company Scope

• Branch Scope

• Warehouse Scope

• Department Scope

• Project Scope

ماژول جدید نباید سیستم Scope جدید ایجاد کند.

۱۱. اصل Event Driven Ready

تمام ماژول‌ها باید قابلیت تولید و مصرف Event داشته باشند.

مثال:

Inventory:

InventoryReceiptCreated

Accounting:

AccountingDocumentCreated

ارتباط بین ماژول‌ها نباید به Query مستقیم دیتابیس وابسته باشد.

۱۲. اصل API و Event Versioning

تمام API‌ها و Event‌ها باید نسخه‌بندی شوند.

مثال:

API:

/api/v۱/orders

Event:

OrderCreated.v۱

تغییر یک ماژول نباید باعث نیاز به تغییر همزمان سایر ماژول‌ها شود.

۱۳. اصل Module Independence

هر ماژول باید بتواند مستقل توسعه، تست و ارتقا پیدا کند.

ارتقای یک ماژول نباید باعث شکستن سایر ماژول‌ها شود.

۱۴. اصل Backward Compatibility

تمام توسعه‌ها باید سازگاری با نسخه‌های قبلی را حفظ کنند.
تغییرات Breaking فقط با برنامه مهاجرت رسمی انجام می‌شوند.

۱۵. اصل Business Rules Separation

منطق کسب‌وکار نباید داخل Database قرار گیرد.

Database مسئول:

- ذخیره داده
- Constraint
- Index
- Integrity

است.

منطق‌هایی مانند:

- محاسبه مالیات
- پورسانت
- تخفیف
- Workflow
- قوانین تایید
- قوانین قیمت‌گذاری

باید در:

Service Layer

پیاده‌سازی شوند.

استفاده از:

- Trigger
- Stored Procedure
- Function

برای Business Logic ممنوع است مگر با تصمیم معماری رسمی.

۱۶. Configuration Before Customization اصل

قبل از توسعه اختصاصی برای یک مشتری باید بررسی شود:

آیا نیاز با موارد زیر قابل حل است؟

- Configuration

- Setting

- Feature Flag

- Workflow

- Rule Engine

اولویت:

Configurable Product

بیش از

Customized Product

است.

۱۷. Reuse Before Create اصل

قبل از ایجاد هر جدول یا سرویس جدید باید بررسی شود:

آیا این قابلیت قبلاً در Core یا Module دیگری وجود دارد؟

ایجاد ساختار تکراری ممنوع است.

۱۸. Single Responsibility اصل

هر:

- جدول

- Service

- Module

- API

باید فقط یک مسئولیت مشخص داشته باشد.

۱۹. Extensible Architecture اصل

معماری باید امکان اضافه شدن قابلیت‌های آینده را بدون تغییر Core فراهم کند.

مانند:

- AI
 - BI
 - Mobile
 - Marketplace
 - IoT
 - BPM
 - E-Commerce
-

۲۰. Architecture Before Coding اصل

قبل از:

- ساخت جدول
- ایجاد API
- طراحی Module
- اضافه کردن Feature

باید جایگاه آن در معماری مشخص شود.

ابتدا معماری، سپس کدنویسی.

نتیجه نهایی معماری

ساختار مرجع پروژه:

Platform



Core



Bounded Context Modules



Services / APIs



Extensions

این سند مرجع نهایی تصمیمات معماری پروژه ERP SaaS است و هر توسعه آینده باید با آن تطبیق داده شود.